In [1]:

```
from preprocess.conversion import visualise_datapoint, visualise_scan_data, convert_all_
```

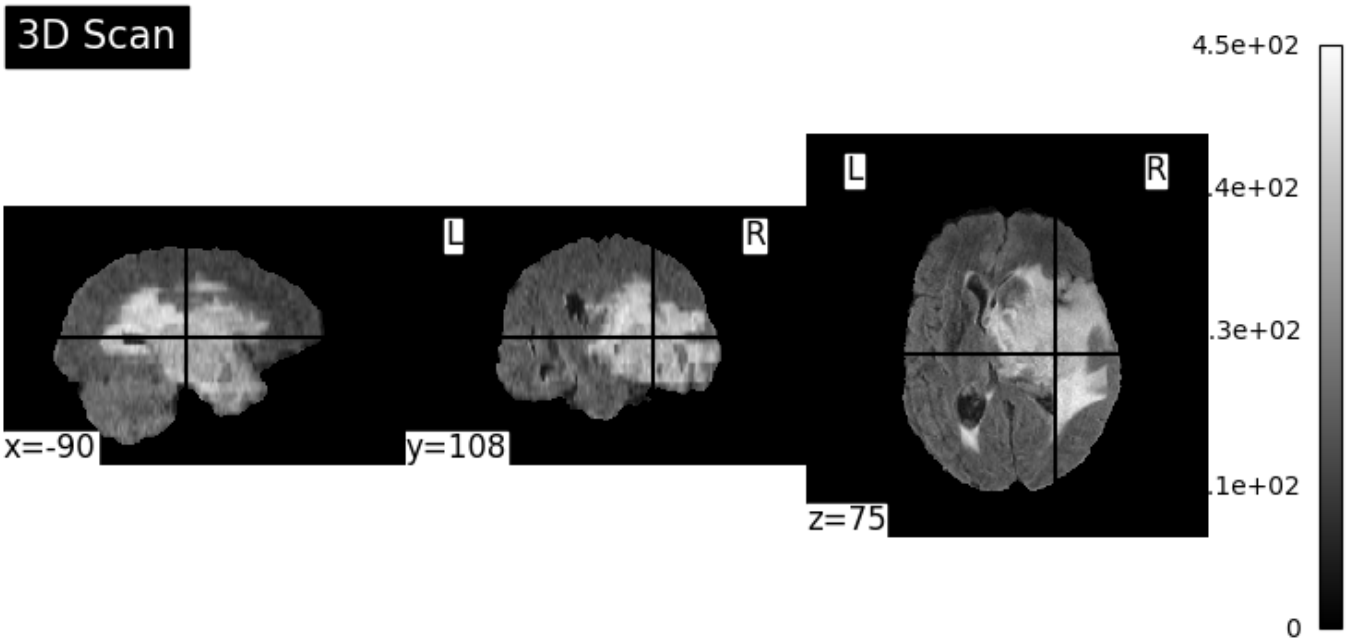
To visualise the samples, I have created several helper functions in preprocess/conversion.py. We can visualise a single datapoint by calling visualise_scan_data(). We can also visualise all four, by calling visualise_datapoint().

In [2]:

```
datapoints = []

datapoints.append("\BraTS20_Training_001\BraTS20_Training_001_flair.nii")
datapoints.append("\BraTS20_Training_001\BraTS20_Training_001_seg.nii")

for datapoint in datapoints:
    visualise_scan_data(datapoint, is_train=True)
```

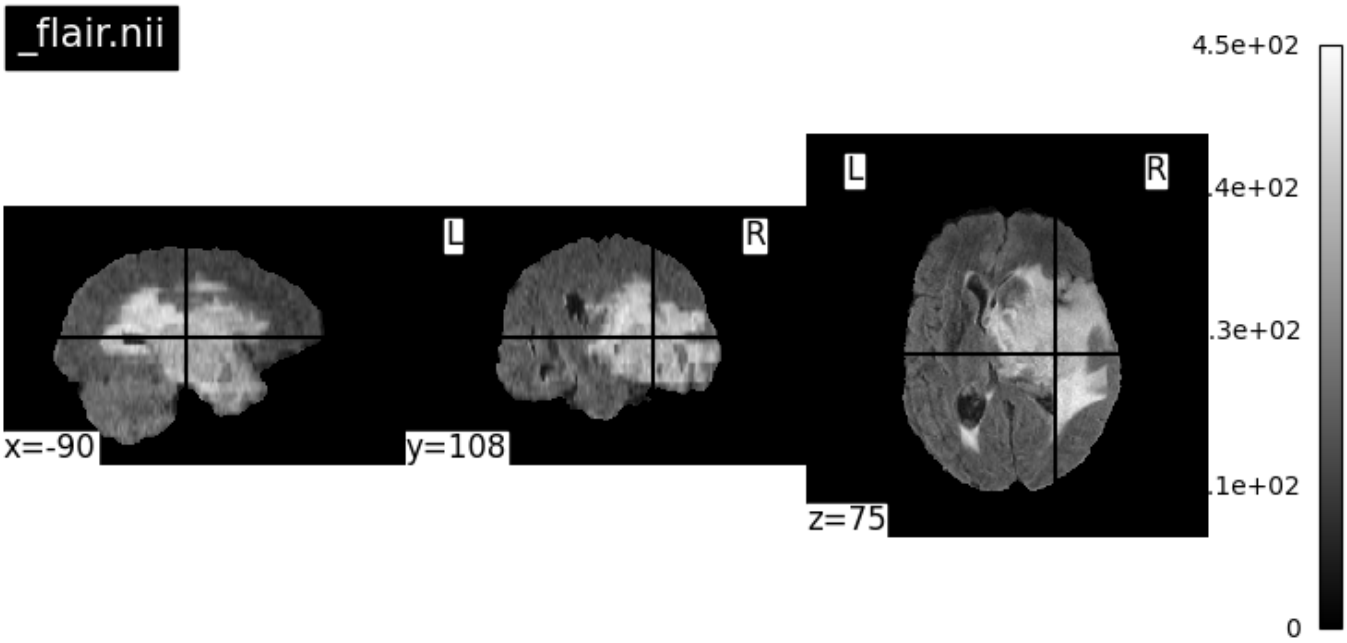


Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

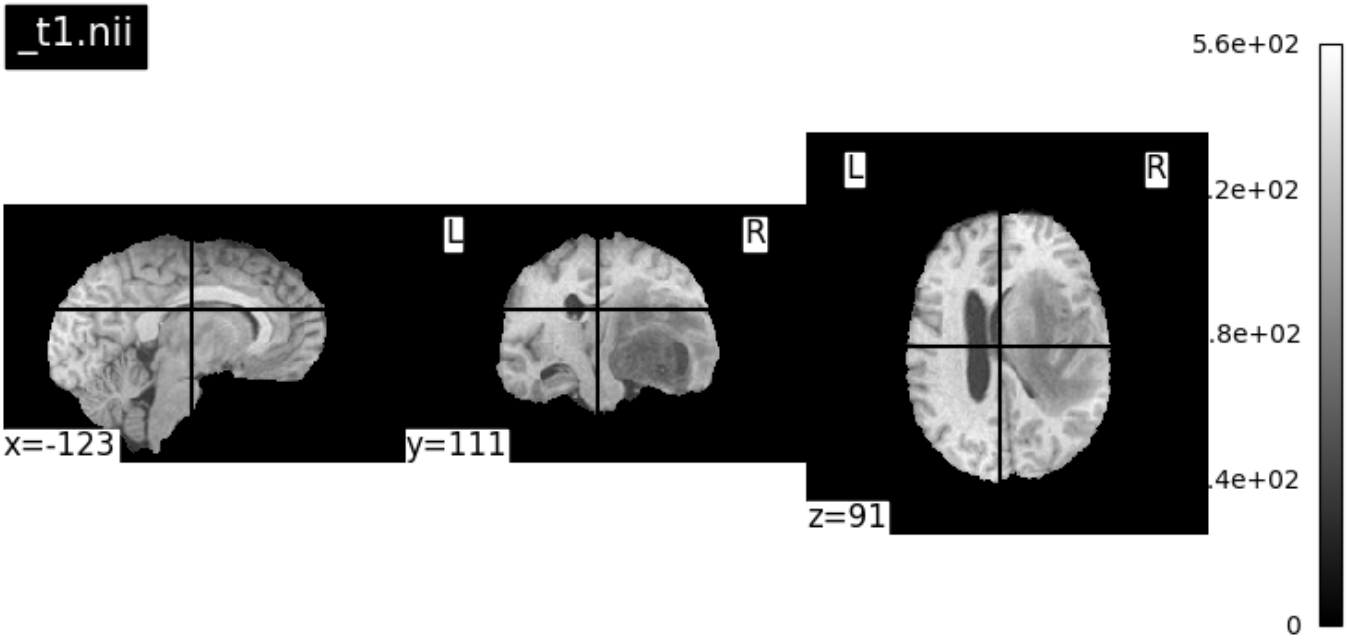
To visualise all four parts, we can use:

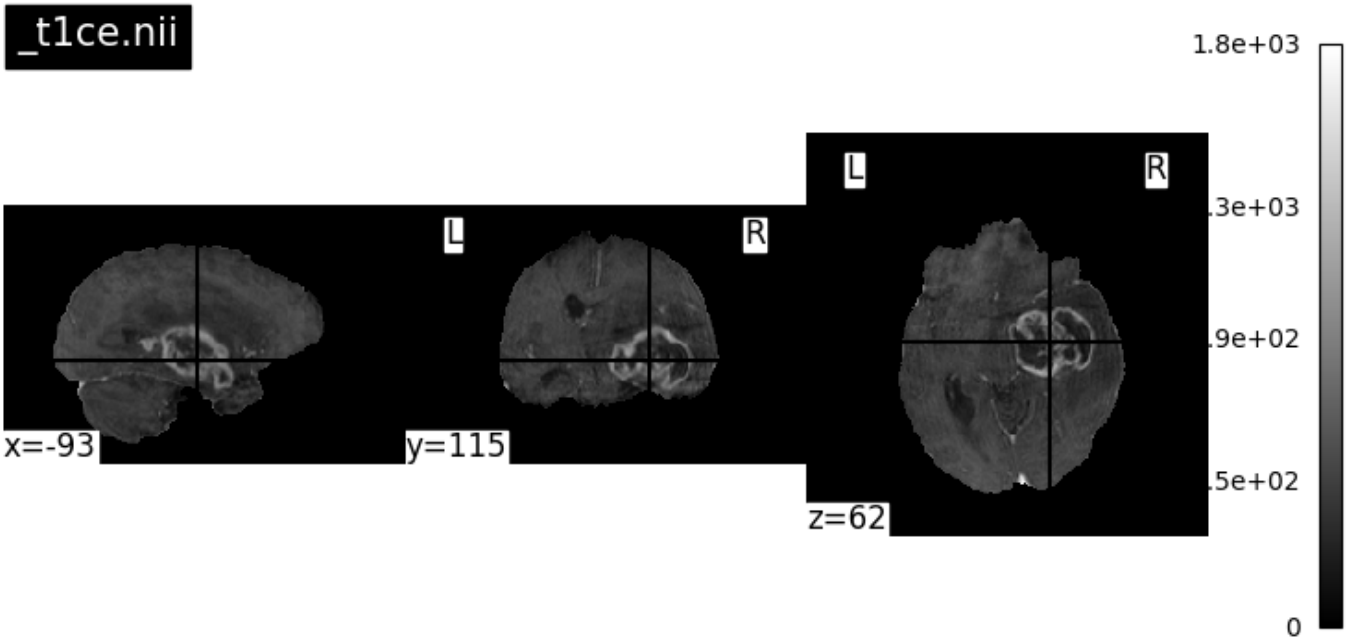
In [3]:

```
path = "\BraTS20_Training_001"
visualise_datapoint(path, is_train=True)
```

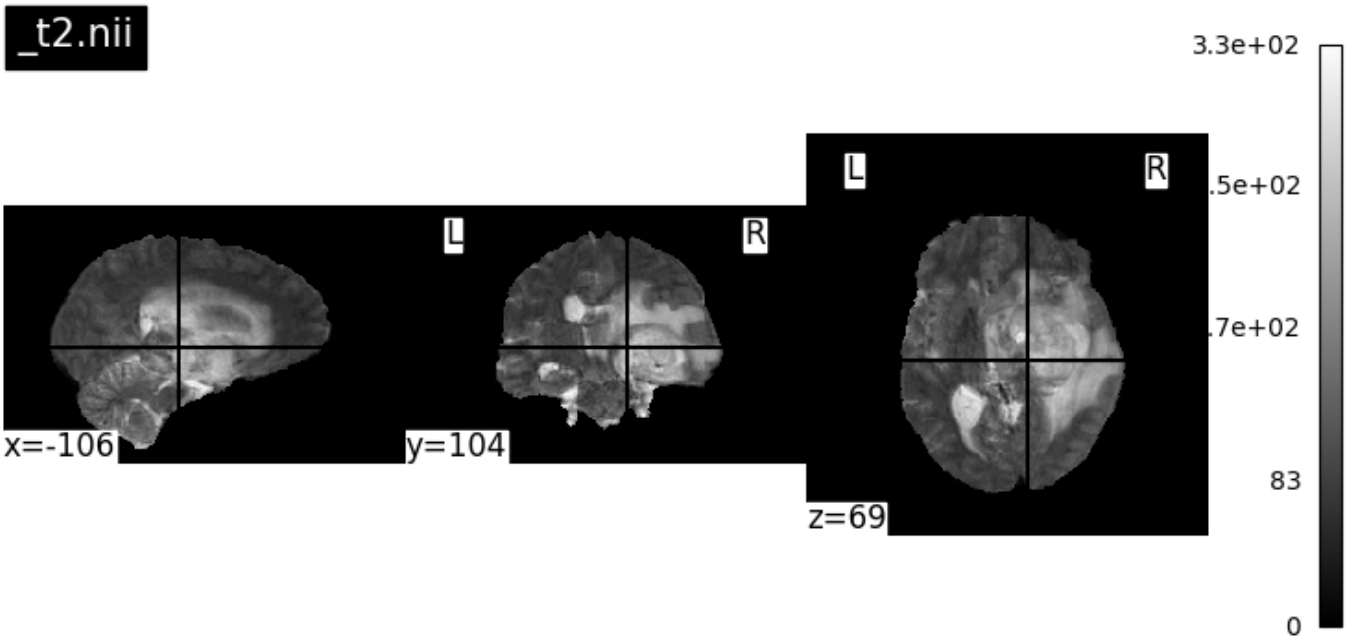


Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF





Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Before training the model, all .nii.gz files should be converted into NumPy arrays and saved locally to significantly reduce I/O overhead during training, since loading compressed medical imaging files repeatedly is slow. As part of this conversion process, each scan must be preprocessed to ensure consistency across the dataset: first, remove empty background regions by cropping to the smallest bounding box containing meaningful information, which reduces memory usage and computational cost. then, we apply scaling using an appropriate scaler, in this case we used z-score normalisation to normalize intensities and stabilize model training.

In [2]:

```
convert_all_npy()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 1
----> 1 convert_all_npy()

NameError: name 'convert_all_npy' is not defined
```

Now that we have completed the converting all the files into numpy format, we have to split the dataset into a training set and a testing set. If we look above, we can see that the t1 image does not look like it provides much useful information. Therefore, it will be skipped and not added into our training pipeline. Now we need to load our data into main memory.

In [1]:

```
from preprocess.splitting import train_test_split, combine_from_npy

features, labels = combine_from_npy()
```

In [2]:

```
features.shape
```

Out[2]:

```
(220, 3, 128, 128, 128)
```

In [3]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
labels.shape
```

Out[3]:

```
(220, 4, 128, 128, 128)
```

In [4]:

```
import torch
if torch.cuda.is_available():
    device = torch.device('cuda')
    print("Using cuda")
else:
    device = torch.device('cpu')
    print("Using cpu")
```

```
Using cuda
```

In [5]:

```
train_dataloader, test_dataloader = train_test_split(features, labels, device)
del features
del labels
```

```
Training batches: 88
Testing batches: 22
```

In [6]:

```
from models.UNet.unet import UNet
from models.UNet.params import UNetParams

params = UNetParams()
model = UNet(params=params).to(device=device)
```

In [7]:

```
from utility.utils import init_weights

model.apply(init_weights)
```

Out[7]:

```
UNet(
  (down_conv1): ConvReLUBlock(
    (conv3d_1): Conv3d(3, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (conv3d_2): Conv3d(16, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 16, eps=1e-05, affine=True)
  )
  (down_conv2): ConvReLUBlock(
    (conv3d_1): Conv3d(16, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (conv3d_2): Conv3d(32, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 32, eps=1e-05, affine=True)
  )
  (down_conv3): ConvReLUBlock(
    (conv3d_1): Conv3d(32, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (conv3d_2): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 64, eps=1e-05, affine=True)
  )
  (down_conv4): ConvReLUBlock(
    (conv3d_1): Conv3d(64, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (conv3d_2): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 128, eps=1e-05, affine=True)
  )
  (bottlenck): ConvReLUBlock(
    (conv3d_1): Conv3d(128, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (conv3d_2): Conv3d(256, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 256, eps=1e-05, affine=True)
  )
  (up_conv1): TransposedConvBlock(
    (conv3d_transposed): ConvTranspose3d(256, 128, kernel_size=(2, 2, 2), stride=(2, 2,
2))
    (conv3d_block): ConvReLUBlock(
      (conv3d_1): Conv3d(256, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1,
1, 1))
      (conv3d_2): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1,
1, 1))
      (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
      (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
      (grp_norm): GroupNorm(8, 128, eps=1e-05, affine=True)
    )
    (groupnorm): GroupNorm(8, 256, eps=1e-05, affine=False)
  )
  (up_conv2): TransposedConvBlock(
    (conv3d_transposed): ConvTranspose3d(128, 64, kernel_size=(2, 2, 2), stride=(2, 2,
2))
    (conv3d_block): ConvReLUBlock(
      (conv3d_1): Conv3d(128, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1,
1, 1))
      (conv3d_2): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
      (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
      (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
      (grp_norm): GroupNorm(8, 64, eps=1e-05, affine=True)
    )
    (groupnorm): GroupNorm(8, 128, eps=1e-05, affine=False)
  )
  (up_conv3): TransposedConvBlock(
    (conv3d_transposed): ConvTranspose3d(64, 32, kernel_size=(2, 2, 2), stride=(2, 2,
2))
    (conv3d_block): ConvReLUBlock(
      (conv3d_1): Conv3d(64, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
      (conv3d_2): Conv3d(32, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
      (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
      (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
      (grp_norm): GroupNorm(8, 32, eps=1e-05, affine=True)
    )
    (groupnorm): GroupNorm(8, 64, eps=1e-05, affine=False)
  )
  (up_conv4): TransposedConvBlock(
    (conv3d_transposed): ConvTranspose3d(32, 16, kernel_size=(2, 2, 2), stride=(2, 2,
2))
    (conv3d_block): ConvReLUBlock(
      (conv3d_1): Conv3d(32, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
      (conv3d_2): Conv3d(16, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
      (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
      (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
      (grp_norm): GroupNorm(8, 16, eps=1e-05, affine=True)
    )
    (groupnorm): GroupNorm(8, 32, eps=1e-05, affine=False)
  )
  (maxpool): MaxPool3d(kernel_size=(2, 2, 2), stride=(2, 2, 2), padding=0, dilation=1, c
eil_mode=False)
  (softmax): Softmax(dim=1)
  (final_conv): Conv3d(16, 4, kernel_size=(1, 1, 1), stride=(1, 1, 1))
)
```

In [8]:

```python
import torch._dynamo
print(torch._dynamo.list_backends())
```

```
['cudagraphs', 'inductor', 'openxla', 'tvm']
```

In [9]:

```python
model = torch.compile(model=model, backend="inductor")
```

In [10]:

```python
from tqdm import tqdm

from utility.utils import train_loop, inference_loop
from utility.metrics import DiceCELoss
epochs = 70

torch.backends.cudnn.benchmark = True
torch.set_float32_matmul_precision("high")

train_losses = []
test_losses = []
train_dice_scores = []
test_dice_scores = []

optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    factor=0.5,
    patience=10,
    min_lr=1e-6
)
loss = DiceCELoss().to(device=device)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```python
for epoch in tqdm(range(1, epochs+1)):

    train_epoch_loss, train_dice_score = train_loop(
        model=model,
        train_dataloader=train_dataloader,
        loss_fn=loss,
        optimizer=optimizer,
        device=device
    )
    test_epoch_loss, test_dice_score = inference_loop(
        model=model,
        test_dataloader=test_dataloader,
        loss_fn=loss,
        device=device
    )

    print(f"training loss : {train_epoch_loss}")
    print(f"testing loss : {test_epoch_loss}")

    train_losses.append(train_epoch_loss)
    test_losses.append(test_epoch_loss)
    train_dice_scores.append(train_dice_score)
    test_dice_scores.append(test_dice_score)
```

```
Training Loop: 88it [05:07,  3.50s/it]
Evaluation Loop: 22it [00:07,  3.07it/s]
  1%|          | 1/70 [05:16<6:04:15, 316.75s/it]
training loss : 2.836289256811142
testing loss : 2.2744785113768144
Training Loop: 88it [05:00,  3.41s/it]
Evaluation Loop: 22it [00:06,  3.31it/s]
  3%|          | 2/70 [10:23<5:52:35, 311.11s/it]
training loss : 2.110939636826515
testing loss : 2.008094560016285
Training Loop: 88it [04:59,  3.40s/it]
Evaluation Loop: 22it [00:06,  3.33it/s]
  4%|          | 3/70 [15:29<5:44:48, 308.78s/it]
training loss : 1.9118406664241443
testing loss : 1.846594209020788
Training Loop: 88it [04:59,  3.41s/it]
Evaluation Loop: 22it [00:06,  3.33it/s]
  6%|          | 4/70 [20:36<5:38:38, 307.86s/it]
training loss : 1.7894079292362386
testing loss : 1.7798011953180486
Training Loop: 88it [05:00,  3.41s/it]
Evaluation Loop: 22it [00:06,  3.33it/s]
  7%|          | 5/70 [25:43<5:33:14, 307.60s/it]
training loss : 1.7035121904178099
testing loss : 1.70496328310533
Training Loop: 88it [05:00,  3.41s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
  9%|          | 6/70 [30:50<5:27:57, 307.47s/it]
training loss : 1.635739654302597
testing loss : 1.6987748633731494
Training Loop: 88it [05:00,  3.41s/it]
Evaluation Loop: 22it [00:06,  3.34it/s]
 10%|          | 7/70 [35:57<5:22:40, 307.31s/it]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
training loss : 1.6118728857148776
testing loss : 1.6165033362128518
Training Loop: 88it [05:00,  3.41s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
 11%|▉         | 8/70 [41:04<5:17:26, 307.21s/it]
training loss : 1.5399180799722672
testing loss : 1.6121694987470454
Training Loop: 88it [05:00,  3.41s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
 13%|▉         | 9/70 [46:11<5:12:18, 307.19s/it]
training loss : 1.4846831438216297
testing loss : 1.5180056636983699
Training Loop: 88it [04:58,  3.40s/it]
Evaluation Loop: 22it [00:06,  3.35it/s]
 14%|▉         | 10/70 [51:17<5:06:40, 306.67s/it]
training loss : 1.4449199560013684
testing loss : 1.5317536809227683
Training Loop: 88it [04:59,  3.41s/it]
Evaluation Loop: 22it [00:06,  3.34it/s]
 16%|▉         | 11/70 [56:23<5:01:31, 306.64s/it]
training loss : 1.4161774096163837
testing loss : 1.4651397954333911
Training Loop: 88it [05:01,  3.42s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
 17%|▉         | 12/70 [1:01:31<4:56:48, 307.04s/it]
training loss : 1.3597430952570655
testing loss : 1.4324596307494424
Training Loop: 88it [04:59,  3.41s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
 19%|▉         | 13/70 [1:06:38<4:51:31, 306.86s/it]
training loss : 1.350489223545248
testing loss : 1.39925308119167
Training Loop: 88it [05:00,  3.41s/it]
Evaluation Loop: 22it [00:06,  3.34it/s]
 20%|▉         | 14/70 [1:11:45<4:46:26, 306.90s/it]
training loss : 1.341614304618402
testing loss : 1.3538544990799644
Training Loop: 88it [04:59,  3.40s/it]
Evaluation Loop: 22it [00:06,  3.35it/s]
 21%|▉         | 15/70 [1:16:51<4:41:07, 306.68s/it]
training loss : 1.2913156551393596
testing loss : 1.3360792073336514
Training Loop: 88it [05:00,  3.41s/it]
Evaluation Loop: 22it [00:06,  3.35it/s]
 23%|▉         | 16/70 [1:21:58<4:36:04, 306.74s/it]
training loss : 1.273972057483413
testing loss : 1.41217271306298
Training Loop: 88it [05:06,  3.48s/it]
Evaluation Loop: 22it [00:08,  2.58it/s]
 24%|▉         | 17/70 [1:27:13<4:33:05, 309.16s/it]
training loss : 1.2395097796212544
testing loss : 1.3232532035220752
Training Loop: 88it [05:47,  3.95s/it]
Evaluation Loop: 22it [00:07,  2.88it/s]
 26%|▉         | 18/70 [1:33:08<4:39:59, 323.07s/it]
training loss : 1.2132777023044499
testing loss : 1.2673404135487296
Training Loop: 88it [05:59,  4.08s/it]
Evaluation Loop: 22it [00:07,  2.89it/s]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
 27%|□□□        | 19/70 [1:39:15<4:45:49, 336.27s/it]
training loss : 1.1665756492452188
testing loss : 1.238920425826853
Training Loop: 88it [05:52,  4.01s/it]
Evaluation Loop: 22it [00:07,  2.80it/s]
 29%|□□□        | 20/70 [1:45:16<4:46:22, 343.65s/it]
training loss : 1.1342543119733983
testing loss : 1.240993843837218
Training Loop: 88it [05:52,  4.01s/it]
Evaluation Loop: 22it [00:07,  2.85it/s]
 30%|□□□        | 21/70 [1:51:17<4:44:50, 348.78s/it]
training loss : 1.1626172343438321
testing loss : 1.22280123017051
Training Loop: 88it [05:51,  4.00s/it]
Evaluation Loop: 22it [00:07,  2.89it/s]
 31%|□□□        | 22/70 [1:57:16<4:41:38, 352.05s/it]
training loss : 1.1264755007895557
testing loss : 1.2198233279314907
Training Loop: 88it [05:59,  4.09s/it]
Evaluation Loop: 22it [00:07,  2.90it/s]
 33%|□□□        | 23/70 [2:03:24<4:39:24, 356.69s/it]
training loss : 1.0921961204572157
testing loss : 1.1598973734812303
Training Loop: 88it [05:54,  4.03s/it]
Evaluation Loop: 22it [00:07,  2.85it/s]
 34%|□□□        | 24/70 [2:09:26<4:34:42, 358.32s/it]
training loss : 1.0636891302737324
testing loss : 1.2379602925343947
Training Loop: 88it [05:54,  4.03s/it]
Evaluation Loop: 22it [00:07,  2.81it/s]
 36%|□□□        | 25/70 [2:15:29<4:29:44, 359.65s/it]
training loss : 1.1104135059497573
testing loss : 1.2620573964985935
Training Loop: 88it [05:50,  3.98s/it]
Evaluation Loop: 22it [00:06,  3.33it/s]
 37%|□□□        | 26/70 [2:21:25<4:23:05, 358.76s/it]
training loss : 1.1183416193181819
testing loss : 1.161387568170374
Training Loop: 88it [05:06,  3.48s/it]
Evaluation Loop: 22it [00:06,  3.34it/s]
 39%|□□□        | 27/70 [2:26:38<4:07:13, 344.96s/it]
training loss : 1.070636349645528
testing loss : 1.1247091510079124
Training Loop: 88it [05:10,  3.53s/it]
Evaluation Loop: 22it [00:06,  3.33it/s]
 40%|□□□        | 28/70 [2:31:55<3:55:34, 336.54s/it]
training loss : 1.0315812534906648
testing loss : 1.1151091077110984
Training Loop: 88it [05:03,  3.45s/it]
Evaluation Loop: 22it [00:06,  3.30it/s]
 41%|□□□        | 29/70 [2:37:06<3:44:37, 328.71s/it]
training loss : 1.0095725655555725
testing loss : 1.1994771551002155
Training Loop: 88it [05:05,  3.47s/it]
Evaluation Loop: 22it [00:06,  3.30it/s]
 43%|□□□        | 30/70 [2:42:18<3:35:52, 323.81s/it]
training loss : 1.0144520909948782
testing loss : 1.0605744827877392
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
Training Loop: 88it [05:05,  3.47s/it]
Evaluation Loop: 22it [00:06,  3.34it/s]
 44%|▉▉▉▉      | 31/70 [2:47:30<3:28:14, 320.37s/it]
training loss : 0.9945752790028398
testing loss : 1.092001809315248
Training Loop: 88it [05:03,  3.45s/it]
Evaluation Loop: 22it [00:06,  3.31it/s]
 46%|▉▉▉▉      | 32/70 [2:52:40<3:20:57, 317.31s/it]
training loss : 1.0053292092951862
testing loss : 1.1477720005945726
Training Loop: 88it [05:10,  3.53s/it]
Evaluation Loop: 22it [00:06,  3.30it/s]
 47%|▉▉▉▉      | 33/70 [2:57:58<3:15:44, 317.42s/it]
training loss : 0.969891638241031
testing loss : 1.1120208285071633
Training Loop: 88it [05:14,  3.57s/it]
Evaluation Loop: 22it [00:06,  3.27it/s]
 49%|▉▉▉▉      | 34/70 [3:03:19<3:11:07, 318.55s/it]
training loss : 0.9700865996154872
testing loss : 1.0574147538705305
Training Loop: 88it [05:21,  3.65s/it]
Evaluation Loop: 22it [00:06,  3.29it/s]
 50%|▉▉▉▉▉     | 35/70 [3:08:47<3:07:27, 321.36s/it]
training loss : 0.9294217432087118
testing loss : 1.017577742988413
Training Loop: 88it [05:21,  3.65s/it]
Evaluation Loop: 22it [00:06,  3.30it/s]
 51%|▉▉▉▉▉     | 36/70 [3:14:15<3:03:12, 323.31s/it]
training loss : 0.9092436974698846
testing loss : 1.026543308388103
Training Loop: 88it [05:20,  3.65s/it]
Evaluation Loop: 22it [00:06,  3.28it/s]
 53%|▉▉▉▉▉▉    | 37/70 [3:19:43<2:58:32, 324.63s/it]
training loss : 0.9050268632444468
testing loss : 1.011141682213003
Training Loop: 88it [05:09,  3.52s/it]
Evaluation Loop: 22it [00:06,  3.29it/s]
 54%|▉▉▉▉▉▉    | 38/70 [3:25:00<2:51:52, 322.25s/it]
training loss : 0.8800127601081674
testing loss : 0.9827075383879922
Training Loop: 88it [05:03,  3.45s/it]
Evaluation Loop: 22it [00:06,  3.30it/s]
 56%|▉▉▉▉▉▉    | 39/70 [3:30:09<2:44:35, 318.56s/it]
training loss : 0.8478382683613084
testing loss : 0.9476320201700384
Training Loop: 88it [05:04,  3.46s/it]
Evaluation Loop: 22it [00:06,  3.31it/s]
 57%|▉▉▉▉▉▉    | 40/70 [3:35:21<2:38:13, 316.44s/it]
training loss : 0.8282234919342127
testing loss : 0.9480337094176899
Training Loop: 88it [05:04,  3.46s/it]
Evaluation Loop: 22it [00:06,  3.35it/s]
 59%|▉▉▉▉▉▉    | 41/70 [3:40:32<2:32:09, 314.81s/it]
training loss : 0.8270948210900481
testing loss : 0.964399359442971
Training Loop: 88it [05:05,  3.47s/it]
Evaluation Loop: 22it [00:06,  3.34it/s]
 60%|▉▉▉▉▉▉    | 42/70 [3:45:44<2:26:31, 314.00s/it]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
training loss : 0.8277061818675562
testing loss : 0.9113021899353374
Training Loop: 88it [05:06,  3.48s/it]
Evaluation Loop: 22it [00:06,  3.35it/s]
 61%|▉▉▉▉▉▉▉     | 43/70 [3:50:57<2:21:10, 313.72s/it]
training loss : 0.7916058498350057
testing loss : 0.970478431745009
Training Loop: 88it [05:08,  3.50s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
 63%|▉▉▉▉▉▉▉     | 44/70 [3:56:12<2:16:05, 314.04s/it]
training loss : 0.7945421629331328
testing loss : 0.9175150719555941
Training Loop: 88it [05:02,  3.44s/it]
Evaluation Loop: 22it [00:06,  3.31it/s]
 64%|▉▉▉▉▉▉▉     | 45/70 [4:01:21<2:10:14, 312.58s/it]
training loss : 0.7686354362151839
testing loss : 0.8984399871392683
Training Loop: 88it [05:03,  3.45s/it]
Evaluation Loop: 22it [00:06,  3.31it/s]
 66%|▉▉▉▉▉▉▉     | 46/70 [4:06:32<2:04:47, 311.96s/it]
training loss : 0.7546139853921804
testing loss : 0.9627155769955028
Training Loop: 88it [05:06,  3.48s/it]
Evaluation Loop: 22it [00:06,  3.30it/s]
 67%|▉▉▉▉▉▉▉     | 47/70 [4:11:45<1:59:44, 312.38s/it]
training loss : 0.7616497671062296
testing loss : 0.9035855233669281
Training Loop: 88it [05:03,  3.45s/it]
Evaluation Loop: 22it [00:06,  3.28it/s]
 69%|▉▉▉▉▉▉▉     | 48/70 [4:16:55<1:54:20, 311.82s/it]
training loss : 0.7389980324290015
testing loss : 0.8911407373168252
Training Loop: 88it [05:03,  3.44s/it]
Evaluation Loop: 22it [00:06,  3.34it/s]
 70%|▉▉▉▉▉▉▉▉    | 49/70 [4:22:05<1:48:54, 311.18s/it]
training loss : 0.7233630466190252
testing loss : 0.9390911188992587
Training Loop: 88it [05:05,  3.47s/it]
Evaluation Loop: 22it [00:06,  3.30it/s]
 71%|▉▉▉▉▉▉▉▉    | 50/70 [4:27:17<1:43:47, 311.40s/it]
training loss : 0.7132925977083769
testing loss : 0.9263468032533472
Training Loop: 88it [05:09,  3.51s/it]
Evaluation Loop: 22it [00:06,  3.33it/s]
 73%|▉▉▉▉▉▉▉▉    | 51/70 [4:32:33<1:39:01, 312.69s/it]
training loss : 0.6998575688763098
testing loss : 0.8612171032211997
Training Loop: 88it [05:05,  3.48s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
 74%|▉▉▉▉▉▉▉▉    | 52/70 [4:37:45<1:33:48, 312.67s/it]
training loss : 0.6885272149335254
testing loss : 0.8745922310785814
Training Loop: 88it [05:03,  3.45s/it]
Evaluation Loop: 22it [00:06,  3.35it/s]
 76%|▉▉▉▉▉▉▉▉    | 53/70 [4:42:56<1:28:23, 311.96s/it]
training loss : 0.6964132599532604
testing loss : 0.9384979361837561
Training Loop: 88it [05:03,  3.45s/it]
Evaluation Loop: 22it [00:06,  3.36it/s]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
 77%|▉▉▉▉▉▉▉▉  | 54/70 [4:48:06<1:23:01, 311.32s/it]
training loss : 0.7039788236672228
testing loss : 0.8744558773257516
Training Loop: 88it [05:05,  3.47s/it]
Evaluation Loop: 22it [00:06,  3.31it/s]
 79%|▉▉▉▉▉▉▉▉  | 55/70 [4:53:17<1:17:52, 311.47s/it]
training loss : 0.7208392762325027
testing loss : 0.9026957181367007
Training Loop: 88it [05:04,  3.46s/it]
Evaluation Loop: 22it [00:06,  3.34it/s]
 80%|▉▉▉▉▉▉▉▉  | 56/70 [4:58:28<1:12:38, 311.30s/it]
training loss : 0.7056422856721011
testing loss : 0.8412742831490256
Training Loop: 88it [05:04,  3.45s/it]
Evaluation Loop: 22it [00:06,  3.31it/s]
 81%|▉▉▉▉▉▉▉▉  | 57/70 [5:03:39<1:07:24, 311.12s/it]
training loss : 0.7037769901481542
testing loss : 0.8810279396447268
Training Loop: 88it [05:03,  3.45s/it]
Evaluation Loop: 22it [00:06,  3.29it/s]
 83%|▉▉▉▉▉▉▉▉  | 58/70 [5:08:49<1:02:11, 310.93s/it]
training loss : 0.6708403835919771
testing loss : 0.8822838013822382
Training Loop: 88it [05:08,  3.50s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
 84%|▉▉▉▉▉▉▉▉  | 59/70 [5:14:04<57:13, 312.12s/it]
training loss : 0.6581282910298217
testing loss : 0.828660330989144
Training Loop: 88it [05:03,  3.45s/it]
Evaluation Loop: 22it [00:06,  3.35it/s]
 86%|▉▉▉▉▉▉▉▉  | 60/70 [5:19:14<51:54, 311.44s/it]
training loss : 0.636467997323383
testing loss : 0.9187384464524009
Training Loop: 88it [05:03,  3.45s/it]
Evaluation Loop: 22it [00:06,  3.31it/s]
 87%|▉▉▉▉▉▉▉▉  | 61/70 [5:24:25<46:40, 311.15s/it]
training loss : 0.6479282409630038
testing loss : 0.8645727228034626
Training Loop: 88it [05:04,  3.46s/it]
Evaluation Loop: 22it [00:06,  3.31it/s]
 89%|▉▉▉▉▉▉▉▉  | 62/70 [5:29:36<41:29, 311.17s/it]
training loss : 0.6474619037048384
testing loss : 0.8246658363125541
Training Loop: 88it [05:04,  3.47s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
 90%|▉▉▉▉▉▉▉▉  | 63/70 [5:34:48<36:19, 311.30s/it]
training loss : 0.6376737799834121
testing loss : 0.8256360265341672
Training Loop: 88it [05:03,  3.44s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
 91%|▉▉▉▉▉▉▉▉  | 64/70 [5:39:57<31:05, 310.85s/it]
training loss : 0.6285010281611573
testing loss : 0.8255294304002415
Training Loop: 88it [05:05,  3.47s/it]
Evaluation Loop: 22it [00:06,  3.31it/s]
 93%|▉▉▉▉▉▉▉▉  | 65/70 [5:45:09<25:55, 311.12s/it]
training loss : 0.6205076107924635
testing loss : 0.8050187826156616
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
Training Loop: 88it [05:07,  3.49s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
 94%|██████████| 66/70 [5:50:23<20:47, 311.96s/it]
training loss : 0.6019291640682654
testing loss : 0.804775129665028
Training Loop: 88it [05:04,  3.46s/it]
Evaluation Loop: 22it [00:06,  3.38it/s]
 96%|██████████| 67/70 [5:55:34<15:35, 311.79s/it]
training loss : 0.5907345979728482
testing loss : 0.7820917774330486
Training Loop: 88it [05:05,  3.47s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
 97%|██████████| 68/70 [6:00:47<10:23, 311.89s/it]
training loss : 0.5802218920805238
testing loss : 0.8000647534023632
Training Loop: 88it [05:01,  3.42s/it]
Evaluation Loop: 22it [00:06,  3.34it/s]
 99%|██████████| 69/70 [6:05:54<05:10, 310.63s/it]
training loss : 0.5633204186504538
testing loss : 0.8011669543656436
Training Loop: 88it [05:00,  3.41s/it]
Evaluation Loop: 22it [00:06,  3.32it/s]
100%|██████████| 70/70 [6:11:01<00:00, 318.02s/it]
training loss : 0.5488698729737238
testing loss : 0.7906155789440329
```

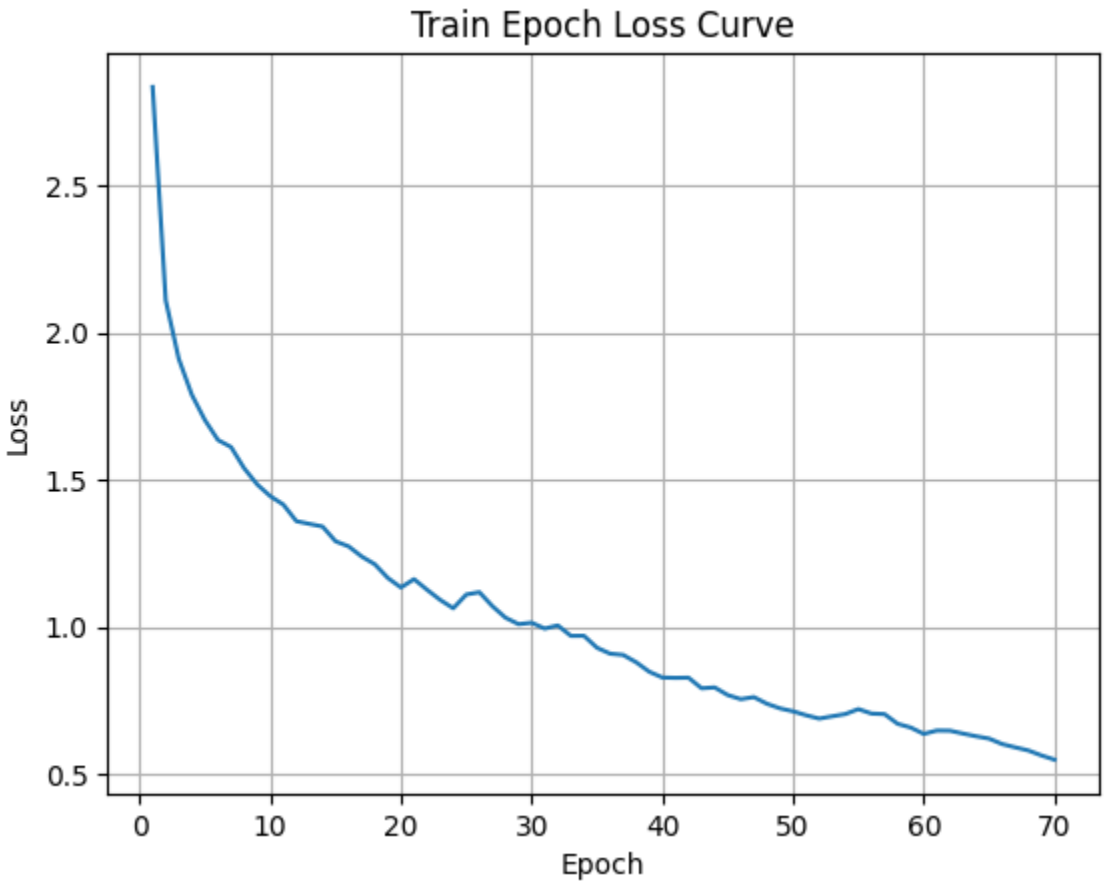
In [11]:

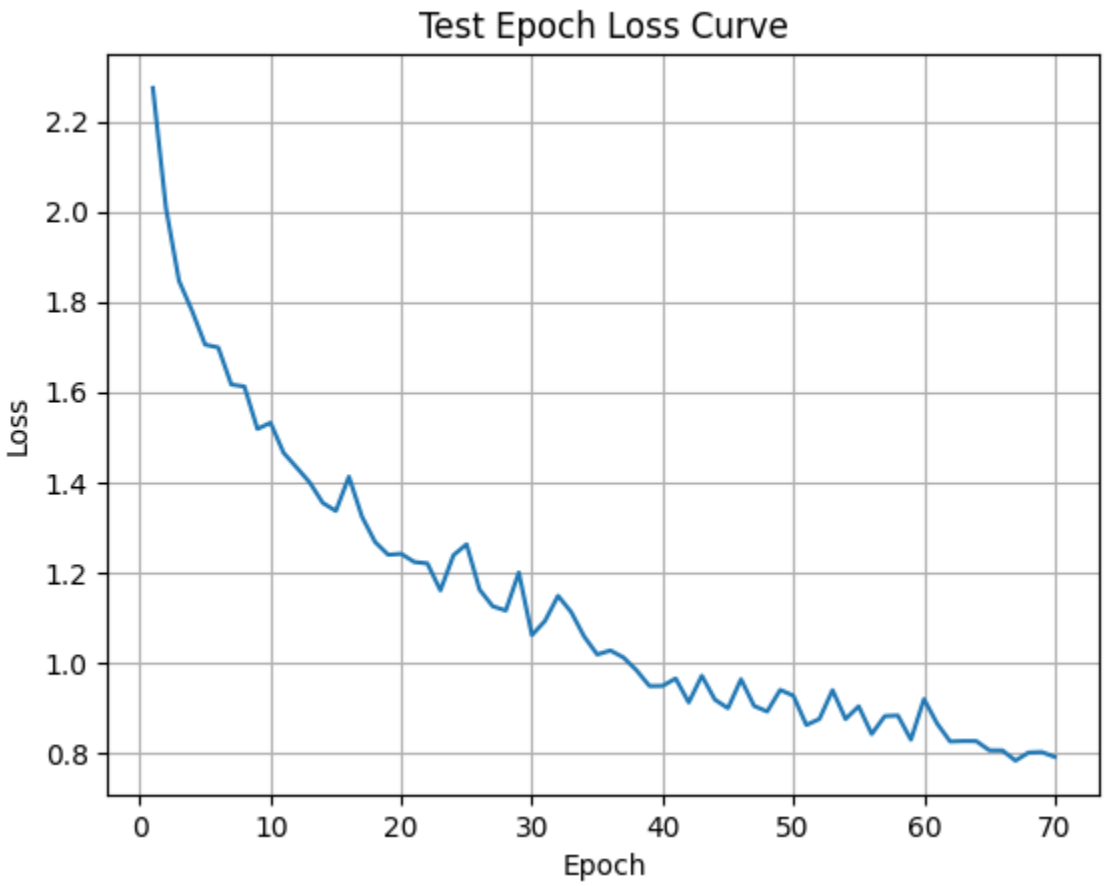
```
torch.save(model.state_dict(), "unet3d_3.1.pth")
del model
del train_dataloader
del test_dataloader
```

In [12]:

```
from utility.plotting import plot_graph
plot_graph(train_losses, "Train Epoch Loss Curve", "Epoch", "Loss")
plot_graph(test_losses, "Test Epoch Loss Curve", "Epoch", "Loss")
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF





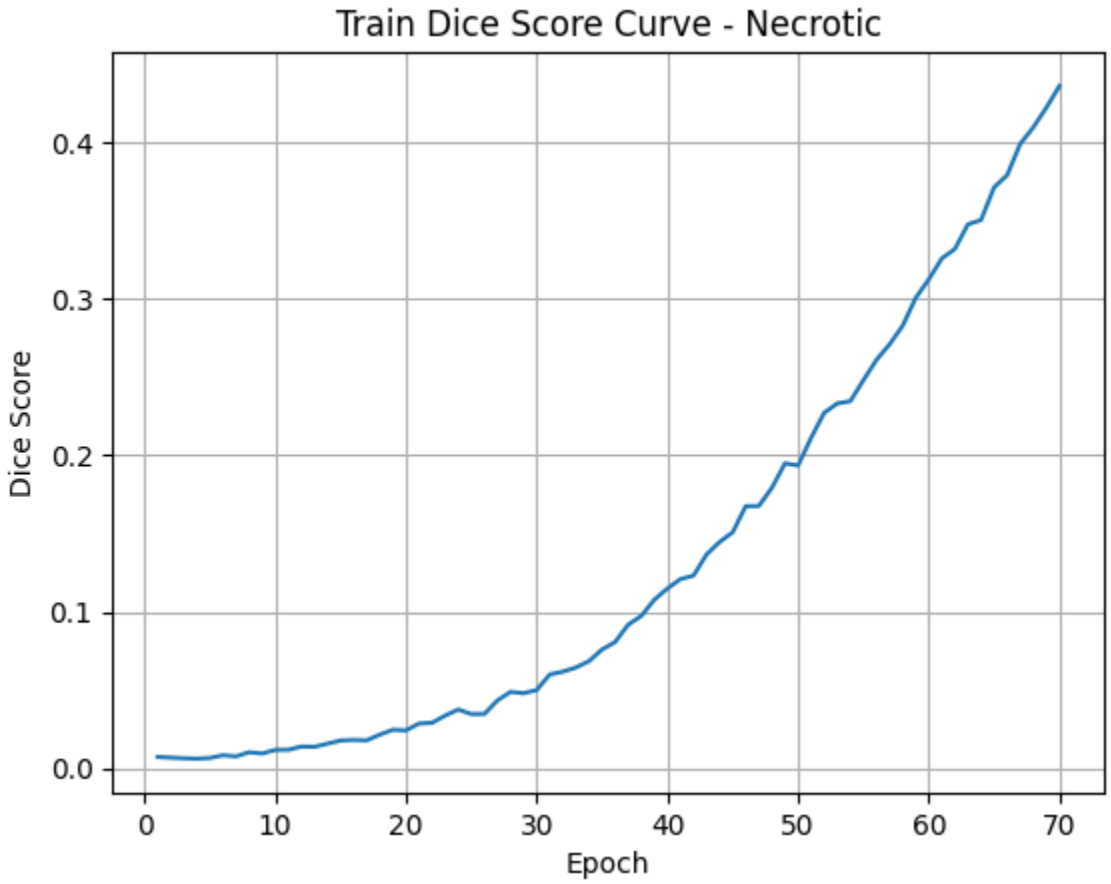
In [13]:

```
plot_graph([t.detach().cpu()[1] for t in train_dice_scores], "Train Dice Score Curve - N
plot_graph([t.detach().cpu()[1] for t in test_dice_scores], "Test Dice Score Curve - Nec
```

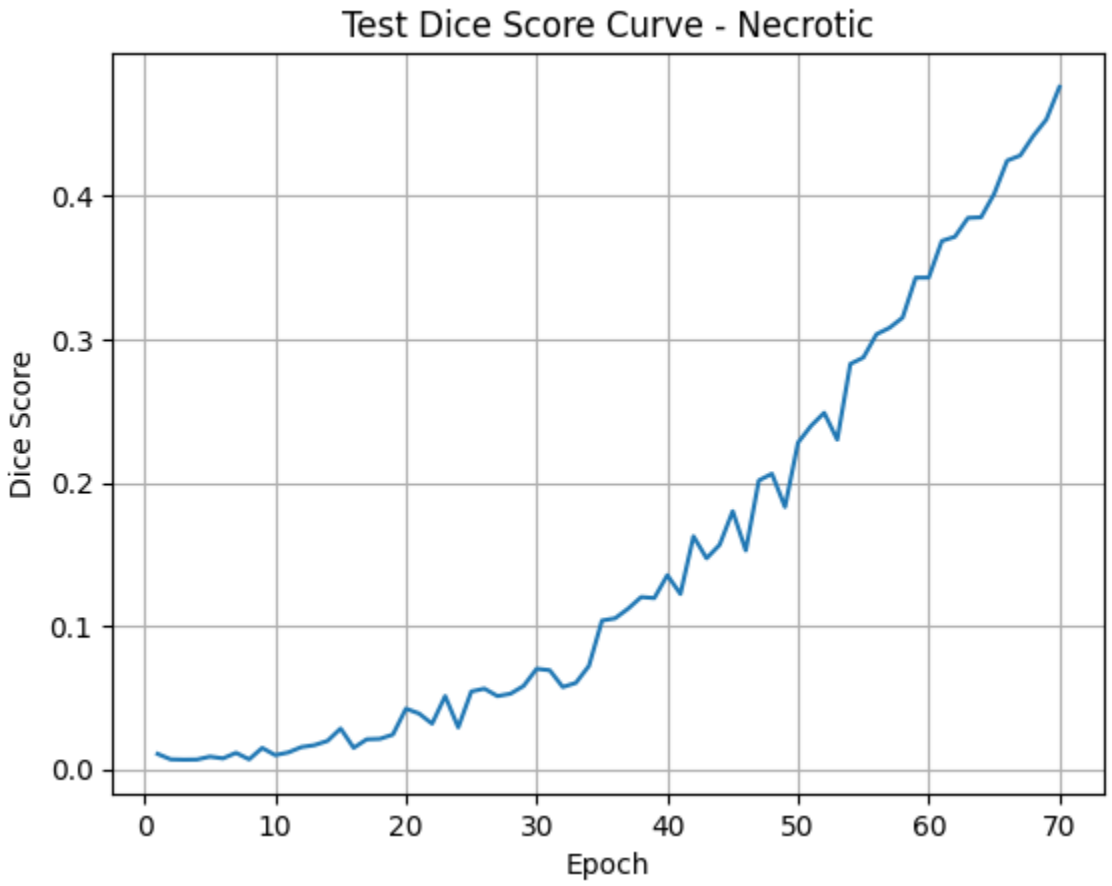
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

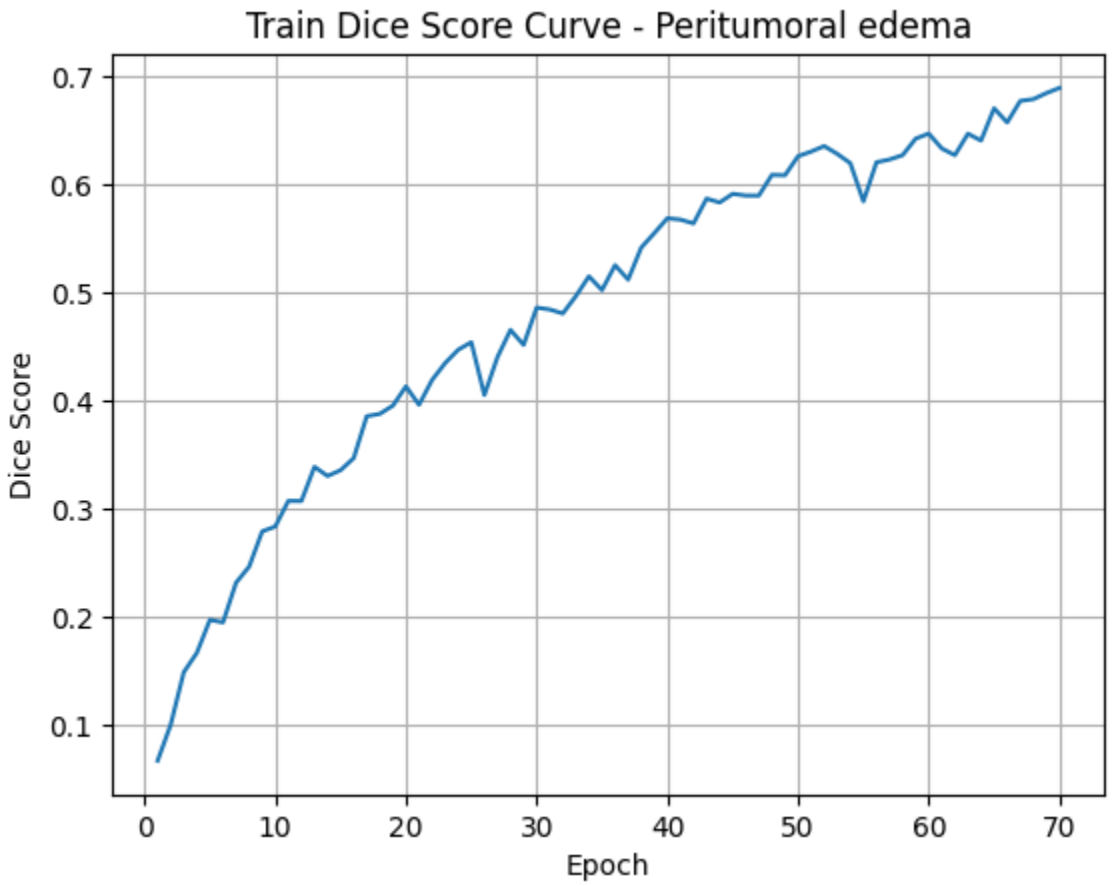
```
plot_graph([t.detach().cpu()[2] for t in train_dice_scores], "Train Dice Score Curve - P
plot_graph([t.detach().cpu()[2] for t in test_dice_scores], "Test Dice Score Curve - Per

plot_graph([t.detach().cpu()[3] for t in train_dice_scores], "Train Dice Score Curve - G
plot_graph([t.detach().cpu()[3] for t in test_dice_scores], "Test Dice Score Curve - GD-
```

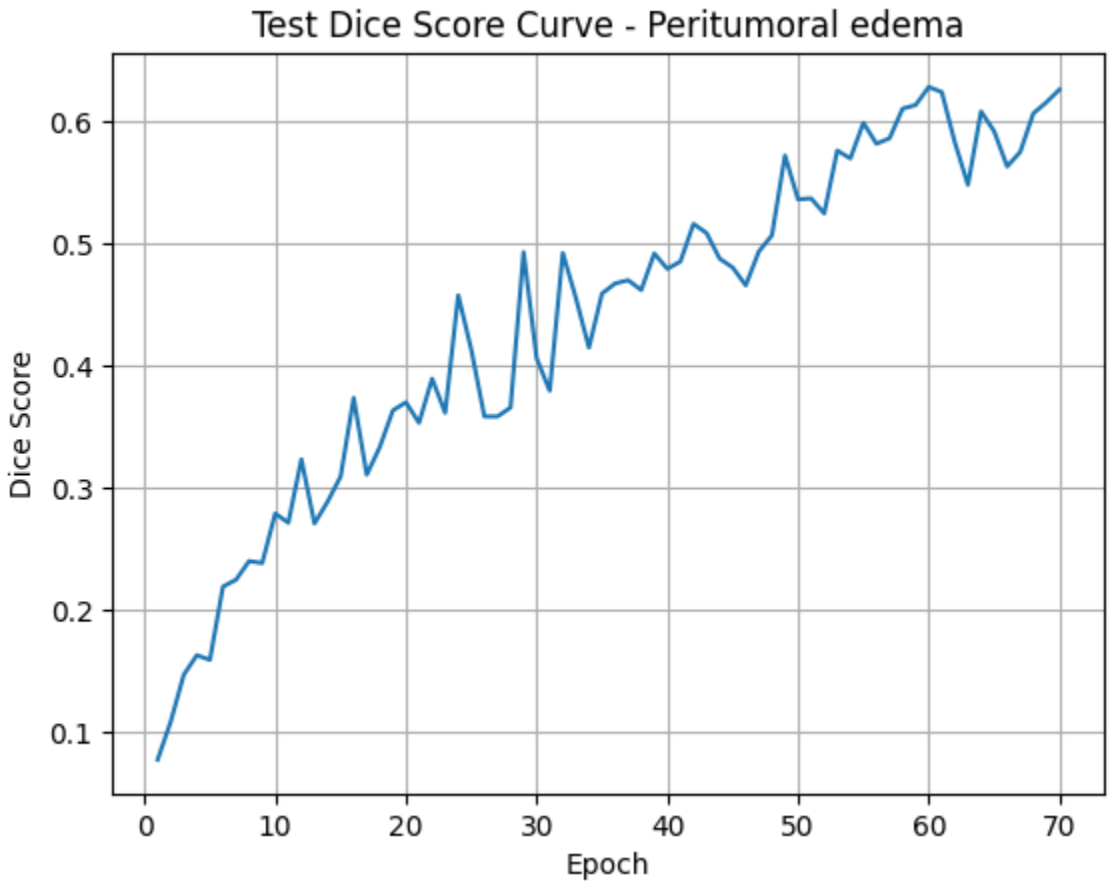


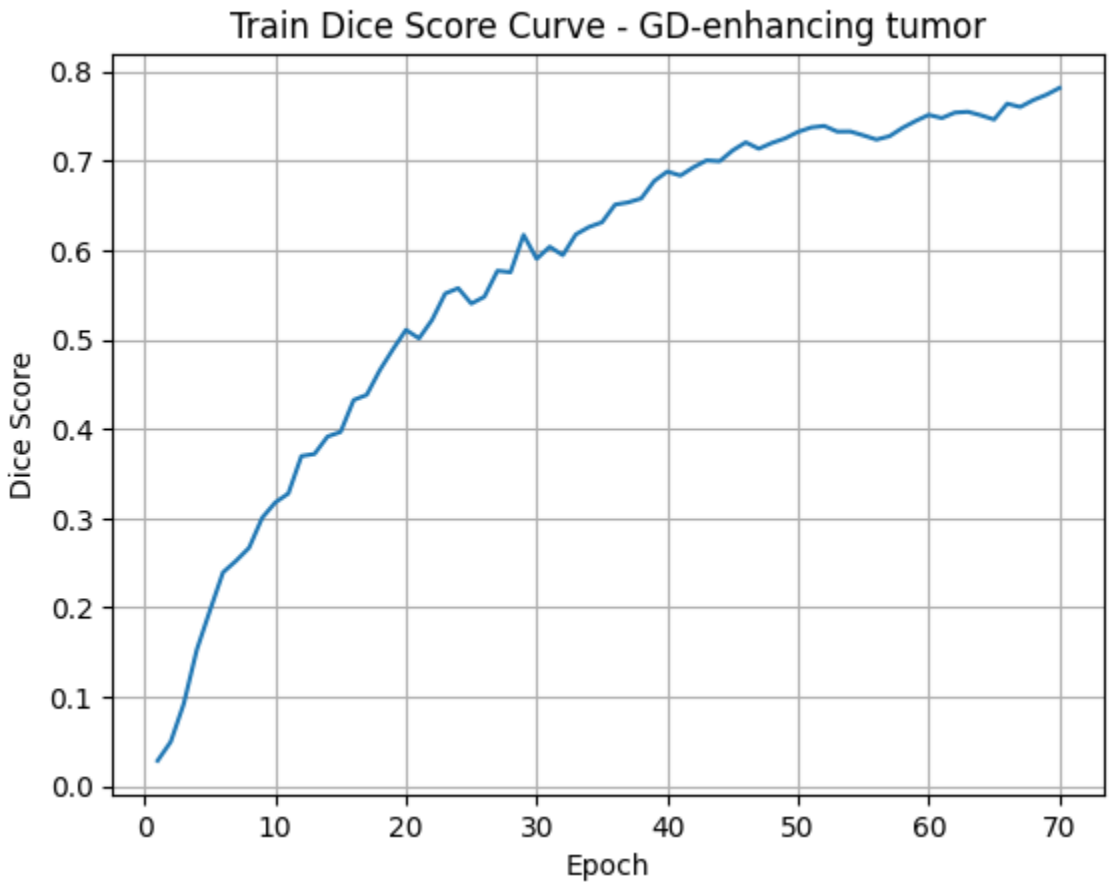
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



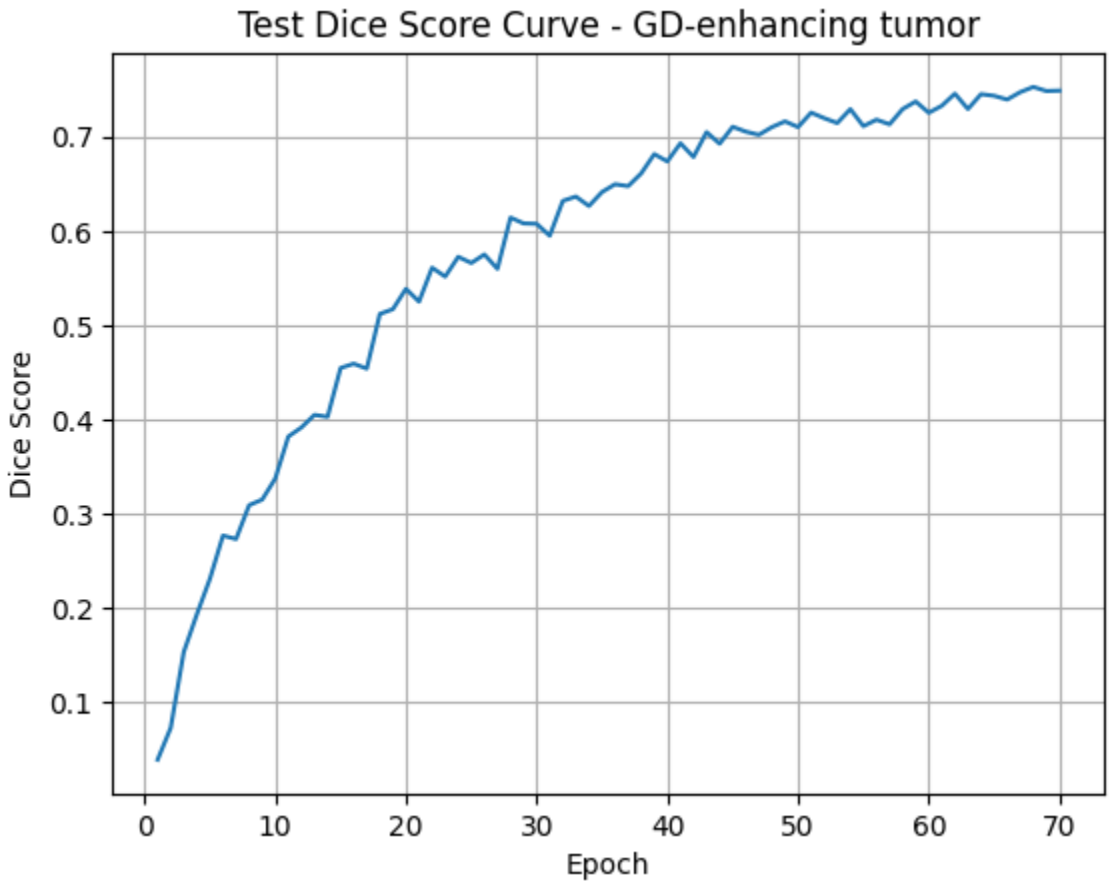


Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF





Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF